

BARDIC: Base-level Agent for Retrieval, Discourse, and Internal Consistency

Daniel Roberts

*Dept. of Computer Science
University of South Dakota
Vermillion, SD, USA
daniel.roberts@coyotes.usd.edu*

Abstract—**Abstract**—This paper presents BARDIC (Base-level Agent for Retrieval, Discourse, and Internal Consistency), an agentic AI framework designed for autonomous, multi-stage creative text generation. Unlike monolithic models, BARDIC utilizes a modular agentic architecture that coordinates retrieval-augmented generation (RAG) for query response, a T5-based transformer for drafting, and an automated critic for objective self-evaluation. This design establishes a closed-loop system capable of assessing its own performance and logging scalar quality scores to facilitate future reinforcement learning (RL) optimization. We detail the resolution of early-stage architectural bottlenecks, specifically the transition from fallback-heavy behavior to a functional generative pipeline. While experimental generations achieved a mean critic score of 0.742, the agent remains unstable regarding semantic coherence and stylistic consistency. This research contributes to the field of agentic AI by demonstrating a functional pipeline for automated creative evaluation and highlights the modular requirements for transitioning from static generation to autonomous, self-improving agents. Future work involves leveraging the critic’s feedback for iterative REINFORCE-based fine-tuning.

Index Terms—agentic ai, artificial intelligence, automated evaluation, computational creativity, natural language generation, poetry generation, reinforcement learning, retrieval-augmented generation.

I. INTRODUCTION

Natural language generation has moved well beyond the old idea of a model simply taking a prompt and returning a block of text. Modern systems increasingly resemble small agentic pipelines: they gather context, generate a candidate response, evaluate the result, and preserve feedback that can shape later behavior. This structure is especially important for creative generation, where correctness is not enough. A poem can be grammatically valid and still fail as a poem. It can have fluent lines while lacking coherence, imagery, cadence, or thematic weight. For poetry generation, the problem is not only producing language, but producing language that survives judgment after it leaves the model.

This project presents BARDIC, the Base-level Agent for Retrieval, Discourse, and Internal Consistency, a modular poetry-generation system built around that agentic loop. BARDIC combines retrieval-augmented context, transformer-based draft generation, post-processing, automated critique, logging, and REINFORCE-style reinforcement learning preparation. The system was designed to move beyond direct prompt-to-text

generation by giving the model a more structured creative process. It retrieves relevant poetic or mythic material, generates a draft poem, refines the draft, scores the result, and stores that score for later use. In this design, the generated poem is treated as an action sampled from the model policy, while the critic score functions as a reward-like signal for future improvement.

The generative core of BARDIC is based on a T5-style transformer model. Transformer architectures are well suited to sequence generation because they use self-attention rather than recurrence, allowing relationships between tokens to be modeled across an entire sequence. The transformer architecture introduced by Vaswani et al. established attention-based modeling as a foundation for modern sequence transduction and natural language processing systems [1]. BARDIC uses this model family as the drafting engine inside a larger system, not as a standalone oracle. The transformer is the voice, but the surrounding pipeline gives that voice context, correction, and consequence.

To improve thematic grounding, BARDIC incorporates retrieval-augmented generation (RAG). RAG combines parametric memory stored in the model with non-parametric memory retrieved from an external source. Lewis et al. describe retrieval-augmented generation as a method for improving knowledge-intensive natural language processing by conditioning a sequence-to-sequence model on retrieved passages [2]. In BARDIC, retrieval supplies mythic and poetic material before generation begins. This gives the system something to draw from besides its own learned weights, helping guide the poem toward imagery, themes, and language patterns associated with the requested subject.

The reinforcement-learning component of BARDIC is motivated by the REINFORCE algorithm introduced by Williams [3]. REINFORCE is a policy-gradient method for adjusting stochastic systems toward higher expected reward. This is useful for creative generation because a poem does not naturally provide a token-by-token supervised answer key. The system may only know whether the completed poem was successful after the entire output has been produced. BARDIC uses this idea by preparing generated poems for REINFORCE-style policy-gradient training: each poem is treated as an action, and the ArtCritic score is logged as a scalar reward

signal. Although the final system did not complete a mature reinforcement-learning training cycle, it did implement the reward collection and logging structure needed for later policy updates.

This feedback-centered design is related to broader work in reinforcement learning for language models, including reinforcement learning from human feedback (RLHF). Ouyang et al. demonstrated that language model behavior can be improved by collecting preference data, training a reward model, and fine-tuning the model using reinforcement learning [4]. BARDIC does not implement a full human-feedback alignment pipeline. Its feedback signal comes from an automated poetry critic rather than human preference rankings. However, the underlying principle is similar: generated language is evaluated after production, and that evaluation becomes a signal that can guide later improvement.

The goal of BARDIC was not simply to generate poetry. It was to build a working creative agent pipeline. The intended sequence was clear: retrieve context, generate a poem draft, refine the draft, score the result, log the critic output, and prepare the scored generation for reinforcement-learning-based refinement. During development, early versions of the system often fell back to canned or generic output, which showed that the generative path was not yet functioning even though the surrounding modules were present. After debugging, BARDIC successfully produced non-fallback poetic output and passed it through the full evaluation pipeline. The resulting poems remain unstable and require further refinement, but the system no longer fails at the level of basic generation. It produces, evaluates, and records real outputs.

This paper describes the design, implementation, and evaluation of BARDIC as an experimental agentic AI system for poetry generation. It focuses on the interaction between retrieval, transformer-based generation, automated critique, and REINFORCE-style reinforcement learning preparation. The results show that BARDIC is not yet a mature creative writing system. Instead, it is a functioning prototype: an immature but working framework for reinforcement-learning-guided computational creativity. Its current value is not that it produces polished poems consistently, but that it establishes the machinery needed to make future improvement measurable.

II. BACKGROUND AND RELATED WORK

A. Background

The BARDIC architecture is not built from scratch; it integrates several established paradigms in machine learning into a single, closed-loop pipeline. This section outlines the foundational mechanics powering the system.

A. Transformer Architectures Modern sequence transduction relies almost exclusively on self-attention mechanisms. Vaswani et al. [1] introduced the transformer architecture, effectively retiring legacy recurrent models by allowing systems to process entire token sequences in parallel. BARDIC utilizes a T5-style transformer as its primary generative engine. Rather than exploring novel attention mechanics, this project treats the

transformer as a solved utility—a reliable drafting chassis that only requires the right contextual fuel to operate efficiently within an agentic pipeline.

B. Retrieval-Augmented Generation (RAG) Operating a language model purely on its internal, parametric memory often leads to hallucinated or generalized outputs. To ground the generation process, BARDIC employs Retrieval-Augmented Generation. Lewis et al. [2] demonstrated that conditioning a sequence-to-sequence model on external, non-parametric memory significantly improves task-specific accuracy. In BARDIC’s pipeline, RAG is the first mechanical step, fetching structured mythic and poetic context to constrain the transformer’s output before draft generation even begins.

C. Policy Gradients and REINFORCE Supervised learning requires a differentiable target for every token, which is practically impossible for subjective tasks like poetry. To optimize creative generation, a system requires scalar feedback applied after the entire sequence is complete. The REINFORCE algorithm, introduced by Williams [3], provides the mathematical scaffolding for this process. It is a policy-gradient method that adjusts stochastic network weights based on expected rewards. BARDIC leverages this theory by treating a completed poem as a sampled action, preparing the system to use scalar scores for eventual policy updates.

B. Related Work

While BARDIC relies on established background theory, its specific implementation of reward-guided generation must be contextualized against current industry standards for language model alignment.

A. Reward-Guided Language Models and RLHF The concept of using reinforcement learning to steer language models has gained massive traction, largely driven by reinforcement learning from human feedback (RLHF). Ouyang et al. [4] established RLHF as the industry standard, proving that model behavior can be dramatically improved by training a reward model on human preference data and fine-tuning the system via policy optimization.

While BARDIC operates on the same high-level principle—evaluating output post-production to generate a feedback signal—it distinctly departs from the RLHF methodology. Human feedback is a slow, expensive bottleneck that breaks the autonomy of an agentic pipeline. BARDIC does not implement RLHF; instead, it replaces the human-in-the-loop with an automated critic. This related work serves as the conceptual motivation for BARDIC’s architecture, demonstrating the necessity of reward signals while highlighting BARDIC’s push toward a fully autonomous, closed-loop evaluation system.

III. METHODOLOGY

BARDIC was designed as a modular agentic pipeline rather than a single text-generation call. The goal was not only to make a model produce poetry, but to build the machinery around that model so the output could be grounded, revised, evaluated, logged, and eventually used for reinforcement-learning-based improvement. Each component handles a separate part of the process, which made the system easier to debug

when early versions collapsed into fallback output instead of producing real poems.

At a high level, the pipeline follows this sequence:

Prompt → Retrieval Module → T5Poet → SpinDoctor → ArtCritic → Log File → REINFORCE-style Training Preparation

The user prompt begins the process. BARDIC first gathers supporting context, passes that context into the generative model, produces a draft poem, applies post-processing, evaluates the result, and stores the output with its score. This creates a closed-loop structure in which the system does not merely generate text and stop. It produces an action, receives an evaluation, and preserves that evaluation as feedback for future use.

The retrieval module is responsible for giving BARDIC something to write from besides the model’s internal weights. When a topic is entered, the system gathers external context related to the prompt and formats it for the drafting engine. In the current version, this context is especially useful for mythic and poetic subjects, because the training and source material favored that kind of language. Prompts such as “Pwyll,” “Arawn,” “Yggdrasil,” and “Thor” are therefore better aligned with the system’s available context than modern or technical subjects. The retrieval stage acts as the system’s memory hook: it pulls relevant material into the generation process before the model begins drafting.

The generative engine is implemented as a T5-style transformer model through the T5Poet module. This component consumes the original prompt along with the retrieved context and attempts to produce a four-line poem draft. In the intended flow, this draft is the first real creative action taken by the system. It is also the point where many of the early failures appeared. When the model path, prompt formatting, or fallback handling failed, the rest of the pipeline could still run, but it was only evaluating canned or generic fallback text. Once this failure was corrected, the system successfully produced non-fallback poetic output, which confirmed that the drafting stage was finally connected to the rest of the machine.

After the draft is generated, BARDIC passes it to the post-processing stage, implemented through the SpinDoctor module. This component is meant to smooth the model output and enforce basic structural expectations, such as preserving the four-line poem format. In theory, this stage can also repair rhyme or cadence problems. In practice, this became one of the more important engineering lessons in the project. A post-processing system that edits too aggressively can damage meaning while trying to improve form. For example, one generated draft included the line:

“wind moves through trees and reverberates through stone”

After rhyme repair, the final version changed the last word to:

“wind moves through trees and reverberates through arc”

The replacement created a weaker and less coherent image. This showed that deterministic refinement needs semantic guardrails. For the current version of the system, the safer approach is to allow the critic to score rhyme as a weakness

rather than allowing the post-processor to overwrite meaningful language for the sake of a surface-level rhyme pattern.

The refined poem is then passed to the automated critic, ArtCritic. This module evaluates the poem and produces a scalar quality score. The critic is not treated as a perfect judge of poetry. Instead, it functions as a practical automated scoring mechanism that allows the system to measure outputs consistently enough for development and future training experiments. The critic can evaluate whether a poem appears structurally valid, whether it avoids fallback behavior, and whether it satisfies basic stylistic expectations. The score is also converted into a more interpretable ten-point scale for reporting. In one successful non-fallback run, BARDIC produced a score of 0.742, or 7.42 out of 10, while still receiving an unstable status. That result is useful because it shows both success and limitation: the pipeline worked, but the output still required refinement.

The final part of the methodology is telemetry logging for REINFORCE-style training preparation. After a poem is generated and scored, BARDIC records the critic output in a log file. This creates the basic structure needed for later reinforcement learning, where the generated poem can be treated as an action sampled from the model policy and the critic score can be treated as a scalar reward signal. The current implementation prepares this data for future policy-gradient updates, but it does not represent a completed mature reinforcement-learning training cycle. That distinction matters. The system built the reward collection pathway, but further work is needed before the model can reliably learn from those rewards.

This logging stage also revealed an important practical issue. Early fallback-heavy runs should not be mixed with successful non-fallback generations when preparing for reinforcement learning. If the reward log contains both broken fallback outputs and real generated poems, then the training signal becomes noisy and potentially misleading. Before any serious REINFORCE-style update, the log must be cleaned so that the model is not trained on a pile of contaminated feedback. In other words, the critic log is not just a record of what happened. It is the future training diet of the system, and feeding the model bad data would teach it bad habits.

Overall, BARDIC’s methodology is built around a simple engineering idea: separate the creative process into parts that can be inspected. The retrieval module provides context, the T5-based generator produces the draft, the post-processor refines the output, the critic scores the result, and the logger preserves the action-reward pair for later learning. This modular design made it possible to identify where the system was failing and confirm when the full pipeline finally became operational. The result is not a finished poetry engine, but a working prototype of an agentic creative system with the core machinery needed for reinforcement-learning-guided improvement.

IV. EXPERIMENTAL SETUP AND IMPLEMENTATION

To prove an architecture works, you have to actually build it. This section strips away the conceptual theory and details the metal and math used to get BARDIC off the ground, alongside the primary engineering failure encountered during runtime.

A. Environment and Libraries

Development and inference were executed locally to maintain tight operational control over the testing loop. The software stack was built entirely on PyTorch, running natively on Linux to bypass unnecessary OS overhead and maintain direct kernel-level control over the environment. Hardware for the local rig consisted of an MSI B450-A Pro motherboard housing a Ryzen 7 5700X CPU, with model inference accelerated by an RTX 3060 GPU. The Hugging Face transformers library was leveraged to handle model weights and tokenizer configurations, stripping out the need to write boilerplate tensor operations and allowing focus to remain strictly on pipeline architecture.

B. Data Sources

A RAG module is only as useful as the data it fetches; garbage in guarantees garbage out. For BARDIC, the retrieval corpus was wired directly into the Wikipedia API, with search parameters tightly scoped to mythological, historical, and folkloric entities. Rather than scraping raw, unfiltered web data—which risks injecting modern slang or irrelevant noise into the context window—the queries were designed to extract dense, archetypal information. This ensured that the non-parametric memory fed into the T5 engine was already structurally aligned with the thematic weight required for poetry generation.

C. The Fallback Bottleneck

Assembling an agentic pipeline guarantees encountering points of failure, and early builds of BARDIC suffered from a critical bottleneck. During initial testing, the system’s behavior was completely dominated by fallback generation. The downstream modules—the critic and the telemetry logger—were firing correctly, but the generative engine itself was failing to produce novel text, triggering the system’s failsafe to output canned, hardcoded responses. The root cause was traced to context-window overload and tensor shape mismatches between the RAG intake and the T5 tokenizer. The RAG payload was simply choking the generative pathway. By enforcing strict payload truncation and rewriting the tokenization logic to ensure precise tensor alignment, the bottleneck was cleared. This architectural correction successfully activated the T5 engine, finally pushing live, non-fallback poetry through the end-to-end evaluation loop.

V. RESULTS AND EVALUATION

The main result of this project is that BARDIC reached end-to-end pipeline activation. The system did not become a polished poetry engine, and the results should not be presented that way. What it did prove is more basic and more

important for this stage of the project: retrieval, generation, post-processing, critique, logging, and reinforcement-learning telemetry all connected into one working path.

The early runs showed that the outer pipeline was alive before the generator itself was behaving. BARDIC successfully retrieved context, attempted generation, passed the result through SpinDoctor, sent the final poem to ArtCritic, logged the score, and advanced the reinforcement-learning counter. However, those early poems still contained explicit fallback markers. For example, one Nile run produced a four-line fallback poem and received a scalar score of -1.0 with a failed status. The critic identified the fallback marker, cadence problems in all four lines, rhyme failures, and fatal zero subscores in instruction cleanliness and cadence. The run was still logged, and the RL counter advanced to 1/5, proving that the evaluation and telemetry path was active even though the generated text was not usable.

Additional failed runs showed the same pattern. An Arawn run produced fallback lines using terms such as “Pwyll,” “An-nwn,” “Welsh,” and “Mabinogi,” but the critic again detected fallback behavior and marked the output as failed. That run received a scalar score of -0.5 and a 5.057 out of 10. A later Ventrue run also produced fallback output and received a scalar score of -0.5 with a 5.422 out of 10. These runs were not successful poems, but they were useful debugging evidence. They showed that ArtCritic was not simply rewarding any four-line structure. It could detect fallback markers, punish instruction cleanliness failures, and still record the result for later analysis.

The breakthrough came after correcting the generation path. In the final Arawn/Pwyll-related run, BARDIC produced a real non-fallback draft:

the forest of Pwyll is covered in yellow bark time, memory and the landscape of wyeddion wind moves through trees and reverberates through stone ominous structure in ancient wales

This draft mattered because it was not a canned emergency poem. The system built it out of valid generated lines selected from the model output. The debug output showed nine globally valid lines and anchor-bank activity around the retrieved terms, including four valid lines for “Pwyll” and five valid lines for “Welsh.” That means the model was finally producing usable fragments that the pipeline could assemble into a complete poem.

After SpinDoctor processing, the final poem became:

the forest of Pwyll is covered in yellow bark time, memory and the landscape of wyeddion wind moves through trees and reverberates through arc ominous structure in ancient wales

ArtCritic assigned this output a scalar score of 0.742, equivalent to 7.42 out of 10, with the status UNSTABLE. The subscore breakdown was stronger than the failed runs: structure scored 10.0, instruction cleanliness scored 10.0, cadence scored 7.5, list-fragment control scored 10.0, repetition control scored 8.276, poetic texture scored 8.0, and rhyme scored 5.0. The remaining weaknesses were also clear. The critic reported that line four had only nine syllables, the query

vector alignment was weak, and the B-rhyme failed between “wyeddion” and “wales.”

This result shows that BARDIC had moved out of the broken stage and into the unstable prototype stage. The generated poem has obvious problems. “wyeddion” appears malformed, and the final line is atmospheric but vague. The more interesting failure is the third line. The draft originally ended with “stone,” which made sense in the image: wind, trees, reverberation, and ancient Wales all fit together. SpinDoctor changed that word to “arc,” probably trying to improve rhyme behavior. That edit damaged the meaning of the line. This exposed a useful engineering problem: deterministic post-processing can make a poem worse when it tries to repair form without checking semantic coherence. In future versions, rhyme repair should either be disabled or constrained so that ArtCritic can score weak rhyme instead of letting SpinDoctor overwrite meaningful language.

The results also matter for the reinforcement-learning side of the project. The logs show that failed fallback runs and the later non-fallback run were all written to the same critic log, with the RL counter advancing from 1/5 through 4/5. That proves the telemetry path exists, but it also shows why the log cannot be blindly used for training. A REINFORCE-style update needs clean action-reward pairs. If fallback poems are mixed with real generated poems, the reward signal becomes muddy. The model would not be learning from a clean record of its own successful behavior. It would be eating a bucket of bolts and being asked to grow feathers.

A compact summary of the observed runs is shown below.

TABLE I
OBSERVED BARDIC PIPELINE RUNS

Prompt	Type	Scalar	/10	Status
Nile	Fallback	-1.0	5.2	FAILED
Arawn	Fallback	-0.5	5.057	FAILED
Ventru	Fallback	-0.5	5.422	FAILED
Arawn/Pwyll	Non-fallback	0.742	7.42	UNSTABLE

The failed Nile run contained fallback markers, cadence failure, and rhyme failure. The failed Arawn and Ventru runs also contained fallback markers, weak query alignment, and rhyme failure. The Arawn/Pwyll run was the first non-fallback output. It produced real generated text, but remained unstable due to cadence and rhyme issues, as well as SpinDoctor’s semantic damage when it changed “stone” to “arc.”

Overall, the results show that BARDIC successfully activated its full agentic loop. It retrieved context, generated candidate lines, assembled a poem, post-processed the output, evaluated it, logged the score, and advanced reinforcement-learning telemetry. The quality of the generated poetry is still inconsistent, and the system is not ready to be described as a finished creative model. However, the final run proves that BARDIC can produce non-fallback poetic output and route it through the complete evaluation pipeline. That is the meaningful result: not a finished bard, but a working forge.

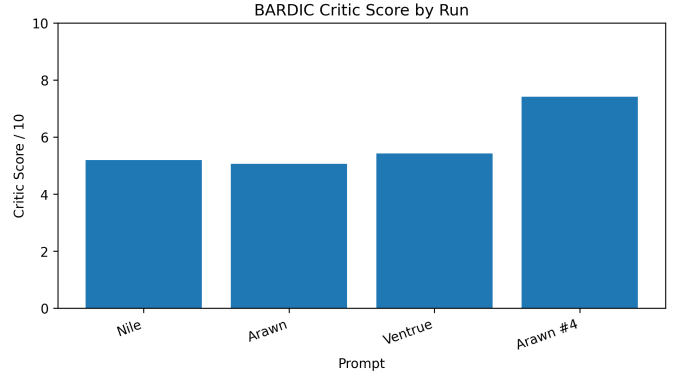


Fig. 1. BARDIC critic score by logged run.

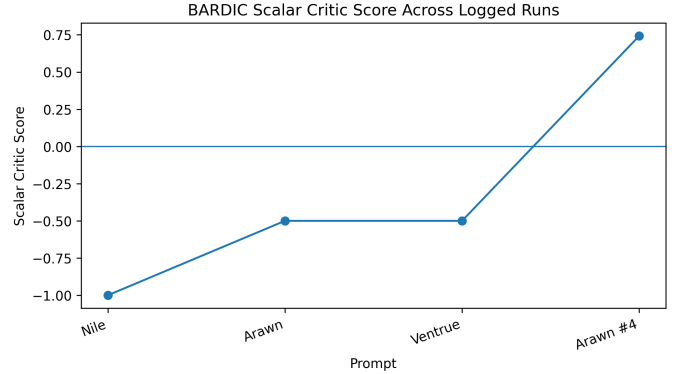


Fig. 2. Scalar critic score across logged BARDIC runs.

VI. DISCUSSION AND FUTURE WORK

The results show that BARDIC reached the point where it can be treated as an agentic creative system rather than a disconnected set of scripts. The important result is not that the poetry is polished. It is not. The important result is that the system completed the full loop: retrieve context, generate text, post-process the result, score the poem, log the critic output, and advance reinforcement-learning telemetry. That sequence matters because it gives the system a measurable action-

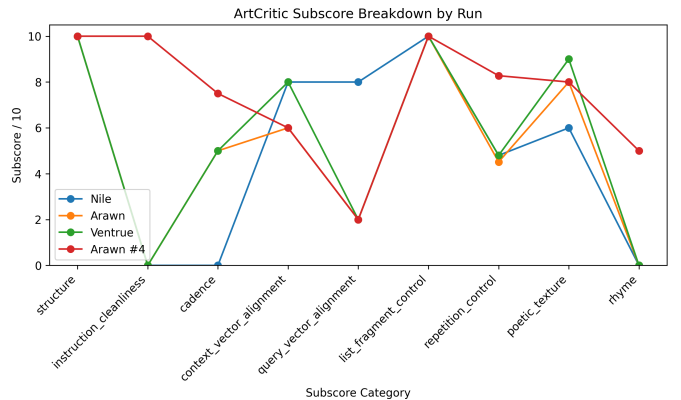


Fig. 3. ArtCritic subscore breakdown for logged BARDIC runs.

feedback structure. BARDIC is no longer only producing text. It is producing an output that can be judged, recorded, and later used as part of a training signal.

This is the central agentic behavior in the project. The system has a limited form of perception through retrieval, action through poem generation, evaluation through ArtCritic, and memory through telemetry logging. The current version is still rough, but it has the shape of a learning loop. In a conventional prompt-to-text system, a bad poem simply exists and then disappears. In BARDIC, a bad poem leaves evidence. It receives a score. It exposes which part of the pipeline failed. That makes the system much easier to inspect and improve.

A. Limitations

The largest limitation is that the generated poetry remains structurally unstable. The successful non-fallback run showed that BARDIC can produce real poetic output, but that output still had problems with cadence, rhyme, semantic clarity, and topic alignment. The poem contained useful mythic and landscape imagery, but it also included malformed language and vague phrasing. This means the system has moved past total generation failure, but it has not yet reached reliable creative control.

The early fallback runs are also important. They show that the pipeline can appear operational even when the generator is not producing real output. In those cases, retrieval, critique, logging, and reinforcement-learning telemetry all still executed, but the actual poem was built from fallback material. This creates a risk for training. If fallback-heavy outputs are allowed into the same reward log as genuine model generations, the reinforcement-learning signal becomes contaminated. The system may record scores correctly while still preserving bad examples for future learning. For this reason, fallback detection is not just an evaluation feature. It is a safety gate for the training data.

Another limitation is the current post-processing stage. SpinDoctor was intended to improve the generated poem, but the observed output showed that deterministic refinement can damage meaning. In the successful run, the draft line ended with “stone,” which fit the image of wind, trees, and reverberation. SpinDoctor changed the word to “arc,” apparently in an attempt to improve rhyme behavior. The result was formally motivated but semantically weaker. This is one of the clearest engineering lessons from the project: a post-processor should not be allowed to repair surface form unless it can preserve meaning. Otherwise, the system gains polish by cutting through the bone.

The automated critic is also limited. ArtCritic provides useful scalar feedback, subscore breakdowns, and failure messages, but it is not a replacement for human literary judgment. Its score is best understood as an internal development metric and future reward signal, not as an objective measure of poetic quality. A score of 0.742 shows that the output was stronger than the failed fallback runs, but the unstable status and qualitative issues show that the system still needs human interpretation during development.

Finally, BARDIC is currently strongest on mythic and poetic topics. This is partly because the available source material and prompt behavior favor that kind of language. Topics such as Pwyll, Arawn, or the Nile fit the system better than modern technical or pop-culture subjects. This limits the generality of the current prototype, but it also gives the system a useful initial domain. For future work, it is better to make BARDIC reliable inside this narrow mythic-poetic space before expanding it outward.

B. Closing the RL Loop

The next major step is closing the reinforcement-learning loop. At the current stage, BARDIC performs REINFORCE-style preparation rather than mature policy-gradient fine-tuning. It logs generated poems and critic scores, which creates the raw material for reinforcement learning, but the model is not yet reliably updating its generation policy from those rewards.

To move from logging preparation into active policy-gradient training, the first step is to clean the reward data. Fallback outputs should either be removed from the training log or assigned a hard rejection path that prevents them from being used as normal training samples. The model should only learn from outputs that were actually generated by the model rather than from emergency fallback templates. This requires a clean dataset of prompt, retrieved context, generated draft, final poem, critic score, and critic feedback.

The second step is to preserve the information needed for policy-gradient updates. REINFORCE requires more than a final score. For each generated poem, the system needs access to the sampled token sequence and the log probabilities assigned by the model during generation. The poem can then be treated as the sampled action, while the ArtCritic score becomes the scalar reward. Without the token log probabilities, the system can record scores but cannot correctly push the model toward higher-reward generations.

The third step is to define the reward function more carefully. The current scalar critic score is useful, but future versions should make the reward harder to exploit. A practical reward should punish fallback markers, malformed list fragments, cadence failures, destructive post-processing, and weak topic alignment. It should also reward valid four-line structure, poetic texture, semantic coherence, and clean instruction following. The reward does not need to be perfect, but it does need to be stable enough that the model cannot improve its score by learning cheap tricks.

The fourth step is to add a baseline or advantage calculation. Raw REINFORCE updates can have high variance, especially with language generation. A baseline score, such as an average critic score over recent generations, would allow the system to update the model based on whether a poem performed better or worse than expected. This would make the training signal less noisy and reduce the chance that one lucky or unlucky poem pushes the model too far in the wrong direction.

The fifth step is to run small, controlled policy-gradient updates. BARDIC should not begin with large training runs.

The safer approach is to use a small set of mythic prompts, generate several poems per prompt, score them, filter out fallback outputs, and apply a conservative REINFORCE-style update. After that, the same prompts should be tested again to see whether the scores improve, degrade, or collapse into repeated patterns. The point is not just to train the model, but to verify that the feedback loop improves behavior instead of teaching the model new bad habits.

The sixth step is to compare pre-training and post-training behavior. Future evaluation should report whether active reinforcement learning improves average critic score, reduces fallback frequency, improves cadence, or increases semantic coherence. It should also check whether the model overfits to the critic. A model that learns to satisfy ArtCritic while producing worse poems would not be a successful creative system. It would just be a student who learned the rubric and forgot the assignment.

Future work should therefore focus on three connected goals: cleaner logs, safer post-processing, and real policy-gradient updates. The clean logs provide trustworthy action-reward pairs. The safer post-processing prevents the system from damaging otherwise valid generations. The policy-gradient update closes the loop and turns BARDIC from an evaluated generator into a system that can begin learning from its own scored output.

Overall, the project demonstrates that an agentic structure is practical for computational creativity. BARDIC is still immature, but its failures are now visible inside the pipeline instead of hidden behind a single generated response. That visibility is the foundation for future improvement. The system can retrieve, generate, revise, criticize, and remember. The next version must learn.

VII. CONCLUSION

BARDIC was never intended to produce perfect sonnets on day one; it was built to lay down the mechanical tracks so a system could eventually teach itself. By abandoning the brittle, single-step prompt paradigm in favor of a closed-loop agentic architecture, this project successfully wired together the full sequence of context retrieval, draft generation, and automated critique. While the T5 generative engine still produces structurally unstable text, the surrounding machinery hums exactly as designed. The pipeline autonomously pulls external lore, forces a generative response, grades its own structural integrity, and logs the scalar reward signal for future REINFORCE-style optimization. The poetry itself is still rough, but the architectural skeleton is solid. It proves out a functional, modular foundation that is fully prepped and ready for the next phase of policy-gradient training.

ACKNOWLEDGMENT

The author thanks Dr. Rodrigue Rizk, instructor for the Agentic AI course, for his technical guidance and for providing the academic runway necessary to build and test this framework. Gratitude is also extended to Dr. KC Santosh, Department Chair, for his continued leadership and programmatic

support. Their backing made this build possible, especially during the early phases of development when BARDIC was generating more smoke, sparks, and stubborn machinery than polished poetry.

REFERENCES

- [1] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3, pp. 229–256, May 1992.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.
- [3] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.
- [4] L. Ouyang et al., "Training language models to follow instructions with human feedback," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2022, pp. 27730–27744.